POSTMAN API TESTING

Advanced Assertions, cURL, Collection Runner and Exporting Collections & Environments

A beginner-friendly practical guide using the supplied Booking API collection

What this guide contains Step-by-step explanations of the uploaded Postman scripts, examples of useful assertions, cURL export/import, running all or selected requests, environment handling, real-world QA scenarios, troubleshooting, and best practices.

Prepared from: Flow(6).txt, Booking API.postman_collection(1).json, and Booking API.postman_environment(1).json

How to Use These Notes

Start with the meaning of assertions, then learn how the supplied JavaScript reads and validates the response. Continue with cURL conversion, Collection Runner execution, and exporting files. Finally, follow the end-to-end practice workflow and complete the exercises.

Section	What you will learn
Part 1 - Advanced assertions	Validate exact values, numeric comparisons, nested JSON fields, response headers, and response time.
Part 2 - cURL	Generate a cURL command from a Postman request and import an existing cURL command.
Part 3 - Collection Runner	Run a complete collection, a folder, or selected requests in a controlled order.
Part 4 - Exporting	Export the collection JSON and environment JSON safely.
Part 5 - Booking API walkthrough	Understand each relevant request and how the variables connect the workflow.
Part 6 - QA practice	Apply the concepts to realistic booking, payment, user, and regression scenarios.

Source fidelity The first explanation of each feature follows the supplied lesson and exported files. Additional recommendations are labelled as best practices or improvements so they are not confused with what the source files explicitly demonstrate.

1. Learning Objectives

After completing this guide, you should be able to:

- Explain what an API assertion is and why a successful HTTP response is not enough by itself.
- Read a JSON response in a Postman test script using pm.response.json().
- Validate a string field, compare a numeric field, and access a nested JSON field.
- Understand the purpose of pm.test(), pm.expect(), and common Chai-style assertions.
- Generate a cURL command from a request and import cURL text into Postman.
- Run all requests, a folder, or selected requests with the Collection Runner.
- Export a collection and an environment, and understand the sensitivity of environment values.
- Identify sequencing and test-data issues in the supplied Booking API collection.

2. What Are API Assertions?

2.1 Meaning

An assertion is a check that compares the actual result from an API with the expected result. In API testing, we use assertions to decide whether the response satisfies the requirement.

Example: You send GET /booking/676. The server returns HTTP 200 and a JSON response. A status-code assertion checks that the status is 200. A body assertion checks that firstname is Abhi. Both checks matter: a 200 response could still contain the wrong customer or incorrect price.

2.2 Common validation layers

Validation	Example check	Why it matters
Status code	Response status is 200	Confirms the HTTP outcome expected by the API contract.
Response body	firstname equals Abhi	Confirms the returned business data.
Data type	totalprice is a number	Prevents unexpected types such as a string.
Nested field	bookingdates.checkin is correct	Validates data inside an object.
Response header	Content-Type indicates JSON	Confirms the response representation.
Response time	Response is below an agreed threshold	Checks a basic performance expectation.

2.3 Where to write the code

Open the request in Postman, choose the Scripts tab, and use the Post-response section (called Tests in older Postman interfaces). This script runs after the server returns a response. It can inspect the response and report pass/fail results in the Test Results area.

3. Understanding the Postman JavaScript API

3.1 pm.test()

```
pm.test("Verify Status Code is 200", function () {
    pm.response.to.have.status(200);
});
```

pm.test() creates a named test. The first argument is a readable test name shown in Postman results. The second argument is a function containing the check. If the assertion passes, Postman marks the test as passed; if it throws an assertion error, the test is marked failed.

3.2 pm.response

pm.response represents the HTTP response received by the current request. It provides access to the status, headers, body, and response time.

3.3 pm.response.json()

```
const jsonData = pm.response.json();
```

This parses a JSON response body and converts it into a JavaScript value. If the response body is a JSON object, jsonData becomes an object whose properties can be read with dot notation. If the response is not valid JSON, parsing can fail; use it only when JSON is expected.

3.4 pm.expect()

```
pm.expect(jsonData.firstname).to.eql("Abhi");
```

pm.expect() starts an assertion. The chained matcher describes the expected condition. In this example, .to.eql("Abhi") checks that the actual value deeply equals the expected value. For primitive strings and numbers, it performs the expected equality check.

3.5 Important note about variable scope

Clarification to a statement in the lesson Flow(6).txt says jsonData was specified globally so it does not need to be defined each time. A variable declared with var, let, or const inside one request script is not automatically shared with

another request script. Define jsonData in each request script where you use it, or deliberately persist data with pm.environment.set(), pm.collectionVariables.set(), or another supported variable scope.

4. Advanced Assertion 1 - Verify First Name

4.1 Code from the lesson

```
const jsonData = pm.response.json();

pm.test("Verify First Name", function () {
    pm.expect(jsonData.firstname).to.eql("Abhi");
});
```

4.2 Explanation line by line

1. const jsonData = pm.response.json(); parses the response body as JSON and stores the result in jsonData.
2. jsonData.firstname reads the firstname property from the response object.
3. pm.test() registers a test with the name Verify First Name.
4. pm.expect(...).to.eql("Abhi") checks that the first name is exactly Abhi.

4.3 Example response

```
{
  "firstname": "Abhi",
  "lastname": "Biradar",
  "totalprice": 874,
  "depositpaid": true
}
```

Expected result: the test passes because firstname is exactly Abhi. If the response contains "Abhishek", the test fails. Exact equality is different from a partial text match.

4.4 Real-time scenario

In a customer-profile API, a QA engineer creates a customer with first name Meera and then calls GET /customers/{customerId}. The test should verify that the returned first name equals Meera. This checks that the service persisted and returned the intended customer data.

4.5 Better defensive version

```
pm.test("Response contains the expected first name", function () {
    const body = pm.response.json();
    pm.expect(body).to.have.property("firstname");
    pm.expect(body.firstname).to.be.a("string");
    pm.expect(body.firstname).to.eql("Abhi");
});
```

This version first verifies that the field exists and is a string. If the API accidentally omits firstname or returns a number, the test gives a more useful failure.

5. Advanced Assertion 2 - Verify Total Price Greater Than 800

5.1 Code from the lesson

```
pm.test("Verify Total Price Greater Than 800", function () {
    pm.expect(jsonData.totalprice).is.greaterThan(800);
});
```

5.2 Explanation

jsonData.totalprice reads the totalprice value from the response. greaterThan(800) checks that the number is strictly greater than 800. A value of 874 passes. A value of 800 or 799 fails because the condition is not greater than 800.

5.3 Use a numeric type check

```
pm.test("Total price is a number and is greater than 800", function () {
    const body = pm.response.json();
    pm.expect(body.totalprice).to.be.a("number");
    pm.expect(body.totalprice).to.be.greaterThan(800);
});
```

5.4 Real-time scenario

An e-commerce order API returns orderTotal. If a test scenario intentionally adds products worth more than Rs. 800, the test can verify that orderTotal is greater than 800. In production-quality tests, calculate or define the expected total from the test data rather than relying on a broad threshold alone.

Boundary testing Test values around the limit: 799, 800, and 801. This helps confirm whether the business rule is greater than 800 (> 800) or greater than or equal to 800 (>= 800).

6. Advanced Assertion 3 - Verify a Nested Check-in Date

6.1 Code from the lesson

```
pm.test("Verify Checkin Date", function () {
    pm.expect(jsonData.bookingdates.checkin).to.eql("2016-04-22");
});
```

6.2 Understand nested JSON

```
{
  "firstname": "Abhi",
  "bookingdates": {
    "checkin": "2016-04-22",
    "checkout": "2024-04-08"
  }
}
```

bookingdates is an object inside the top-level response. checkin is a property inside bookingdates. JavaScript dot notation therefore reads it as jsonData.bookingdates.checkin.

6.3 Explanation

1. jsonData accesses the parsed response object.
2. bookingdates moves into the nested bookingdates object.
3. checkin reads the check-in date string.
4. .to.eql("2016-04-22") compares it with the expected date.

6.4 Defensive nested-property assertion

```
pm.test("Check-in date is correct", function () {
    const body = pm.response.json();
    pm.expect(body).to.have.property("bookingdates");
    pm.expect(body.bookingdates).to.have.property("checkin");
```

```
    pm.expect(body.bookingdates.checkin).to.eql("2016-04-22");
});
```

Checking the parent object and child property first makes failures easier to understand than attempting to access a missing nested object directly.

6.5 Real-time scenario

A hotel reservation API returns checkInDate and checkOutDate inside a stay object. The test should validate both nested dates and ensure the dates match the request submitted by the test.

7. A Complete and Reusable Assertion Script

The following is an improved combined example for the GET Booking Details request. The expected values must match the test data created by the preceding POST request.

```
pm.test("Status code is 200", function () {
    pm.response.to.have.status(200);
});

const body = pm.response.json();

pm.test("Booking has required fields", function () {
    pm.expect(body).to.have.property("firstname");
    pm.expect(body).to.have.property("lastname");
    pm.expect(body).to.have.property("totalprice");
    pm.expect(body).to.have.property("bookingdates");
});

pm.test("First name is correct", function () {
    pm.expect(body.firstname).to.eql("Abhi");
});

pm.test("Total price is numeric and above 800", function () {
    pm.expect(body.totalprice).to.be.a("number");
    pm.expect(body.totalprice).to.be.greaterThan(800);
});

pm.test("Check-in date is correct", function () {
    pm.expect(body.bookingdates.checkin).to.eql("2016-04-22");
});
```

Keep expected data consistent If the POST request uses a variable such as {{fname}}, avoid hard-coding a different first name in the GET assertion. Either use a fixed test value consistently or compare the response with the same expected value stored in a variable.

8. Export a Postman Request as a cURL Command

8.1 What is cURL?

cURL is a command-line tool used to transfer data to or from a URL. It can send HTTP requests with methods, headers, and request bodies. Engineers use it to reproduce an API call from a terminal, share a minimal reproduction with a developer, or run simple checks outside Postman.

8.2 Steps shown in the lesson

1. Open the request in Postman.
2. Choose the Code option or code-generation control for that request.

3. Select cURL from the available language choices.
4. Copy the generated command.

8.3 cURL example from the lesson

```
curl --location 'https://restful-booker.herokuapp.com/booking' \
--header 'Content-Type: application/json' \
--data '{
  "firstname": "Abhi",
  "lastname": "Biradar",
  "totalprice": 874,
  "depositpaid": true,
  "bookingdates": {
    "checkin": "2016-04-22",
    "checkout": "2024-04-08"
  },
  "additionalneeds": "Breakfast and Meals"
}'
```

8.4 Explanation of each part

cURL part	Meaning
--location	Follows HTTP redirects when the server responds with a redirect.
'https://.../booking'	The endpoint URL being called.
--header Content-Type: application/json	Tells the server that the request body is JSON.
--data ...	Sends the supplied data as the request body; cURL uses POST by default with --data unless another method is specified.
JSON object	Contains the booking details to create.

8.5 Why this is useful in real projects

- A developer can reproduce an API issue from a terminal without opening the Postman UI.
- A QA engineer can attach a reproducible command to a bug report.
- A DevOps engineer can run a simple API smoke check from a shell script or pipeline.
- A team can compare whether the same request behaves consistently from Postman and the command line.

Security Do not paste production passwords, bearer tokens, cookies, personal data, or internal hostnames into public bug trackers or chat rooms. Generated cURL commands may contain authentication data.

9. Import a cURL Command into Postman

9.1 Steps from the lesson

1. Click Import in Postman.
2. Choose Raw Text (or the equivalent cURL import option in your Postman version).
3. Paste the complete cURL command.
4. Continue with the import and save the generated request into the desired collection.

9.2 What Postman does

Postman parses the cURL command and attempts to create a matching request: method, URL, headers, and body. Review the imported request before using it. Some shell options, quoting rules, or variables may not map exactly to Postman settings.

9.3 Verification after import

- Confirm the HTTP method is correct.
- Confirm the URL and path are correct.
- Check that all required headers were imported.
- Open Body and verify the JSON is complete and valid.
- Check authentication separately; tokens in a cURL command may be sensitive or expired.
- Send the request against a safe test environment and compare its result.

10. Execute the Entire Collection Using Collection Runner

10.1 What is Collection Runner?

Collection Runner executes requests saved in a collection in a controlled sequence. It can run the requests and their associated scripts, show response results, and report which tests passed or failed. It is useful for repeating a regression suite without manually clicking Send for every request.

10.2 Steps from the lesson

1. Select the Booking API collection.
2. Choose Run / Run collection.
3. Review the requests selected for execution.
4. Keep requests in the intended sequence.
5. Click Start Run and review the run summary and individual request results.

10.3 Why execution order matters

Some requests depend on values created by earlier requests. In this collection, Create Booking stores a booking ID in b_id, and Token Generator stores a token in b_token. GET, PUT, and DELETE requests use these variables. If a dependent request runs before the variable is populated, the URL or Cookie header may contain an empty or stale value.

Step	Request	Dependency / outcome
1	GET Booking Ids	Independent read request; returns available booking IDs.
2	Create Booking	Creates a new booking and stores its bookingid in b_id.
3	GET Booking Details As Per Id	Uses {{b_id}} to retrieve the booking created earlier.
4	Token Generator	Creates an authentication token and stores it in b_token.
5	Update Booking	Uses {{b_id}} and Cookie token={{b_token}} to update the booking.
6	Curl Command	Creates another booking from a static cURL-derived request; it does not update b_id in the supplied collection.

Step	Request	Dependency / outcome
7	Delete Booking	Uses {{b_id}} and {{b_token}} to delete the booking referenced by b_id.

Important observation about the uploaded collection The cURL Command request creates a second booking but does not save that response bookingid into b_id. The final Delete Booking therefore targets the first booking ID stored by Create Booking, not necessarily the booking created by Curl Command. For repeatable cleanup, separate the cURL demonstration from the main lifecycle flow or capture and delete its own ID.

10.4 Suggested lifecycle order

For a clean end-to-end lifecycle test, use this logical order:

1. Generate token (when the next operations require authentication).
2. Create booking and capture the returned booking ID.
3. GET the booking and verify the initial values.
4. PUT the booking and verify the updated values with another GET.
5. DELETE the test booking.
6. Optionally GET the deleted ID and verify the API returns the documented not-found result.

GET Booking Ids can be run independently as a discovery/read check. Keep the cURL-imported duplicate create request in a separate demonstration folder unless its own ID is captured and cleaned up.

11. Run a Specific Folder or Selected Requests

11.1 Run one folder

1. Locate the folder in the collection sidebar.
2. Open the folder menu (three dots).
3. Choose Run.
4. Review the requests and click Start Run.

Use this when you want to run only a functional area, such as Positive Endpoints or Negative Scenario.

11.2 Run selected requests from a folder

1. Open the folder menu and choose Run.
2. Select only the requests that you want to execute.
3. Confirm their order, then start the run.

This is useful when a defect affects only one endpoint or when you want to run a small smoke subset. Be careful: a selected request may depend on a variable generated by a request that you did not select.

11.3 Example

After a change to booking-date validation, run the Create Booking and GET Booking Details requests that are relevant to the change, plus any supporting authentication request required by the flow. If the GET request uses {{b_id}}, either run the create request first or provide a valid test ID safely.

12. Export the Collection as JSON

12.1 Meaning

A Postman collection export is a JSON representation of the collection structure. It can include request names, methods, URLs, headers, bodies, folders, scripts, and other collection metadata. It is useful for version control, sharing a reproducible test suite, and backup.

12.2 Steps from the lesson

1. Open the collection menu (three dots).
2. Choose More or the export option.
3. Choose the collection export format offered by Postman (the lesson uses the v2.1 schema).
4. Click Export and save the JSON file in the required folder.

12.3 What to do with the export

- Store the collection in a controlled project repository if your team uses version control.
- Use meaningful filenames, such as Booking_API.postman_collection.json.
- Review request bodies and scripts before sharing.
- Do not assume that exporting a collection also safely packages every environment or secret needed to run it.

13. Export Environment Variables as JSON

13.1 What is an environment?

A Postman environment stores values that can change between environments or runs. The supplied Booking API environment includes the keys fname, b_id, and b_token. Requests refer to these values using double curly braces, for example {{fname}} or {{b_id}}.

Variable	Purpose in the supplied collection	How it is used
fname	First name used in the Create Booking request body.	{{fname}}
b_id	Booking ID captured from the Create Booking response.	/booking/{{b_id}}
b_token	Authentication token captured from Token Generator.	Cookie: token={{b_token}}

13.2 Export steps from the lesson

1. Select the Booking API environment.
2. Open the environment menu (three dots).
3. Choose Export and save the environment JSON file.

13.3 Critical security point

Do not share the uploaded environment export as-is The supplied environment JSON contains a token value. Treat tokens and credentials as secrets even when they belong to a demo service. The example notes intentionally do not reproduce the token. Before sharing an environment export, remove or replace secret values, use safe placeholders, or use an approved secret-management workflow. Regenerate/revoke a token if it may have been exposed.

13.4 Imported environment values may be stale

The exported environment includes a booking ID and a token value. These can expire, be deleted, or refer to data that no longer exists. The preferred automated flow is to generate a fresh token and create a fresh booking during the run, then save the returned values in the selected environment. Do not rely on old IDs or tokens from an exported file.

14. Walkthrough of the Uploaded Booking API Collection

This section connects the concepts to the actual request names and configuration in Booking API.postman_collection(1).json. The source collection contains Positive Endpoints and Negative Scenario folders.

14.1 GET Booking Ids

Setting	Value
Method	GET
URL	https://restful-booker.herokuapp.com/booking
Test	Verify Status Code is 200

```
pm.test("Verify Status Code is 200", function () {
    pm.response.to.have.status(200);
});
```

Purpose: retrieve the list of booking identifiers. This request is useful for discovery and checking that the booking endpoint is responding. The supplied collection validates the HTTP status; a stronger test could also validate that the response is an array and that each item contains a bookingid.

```
pm.test("Response is an array", function () {
    const body = pm.response.json();
    pm.expect(body).to.be.an("array");
});
```

14.2 Create Booking

Setting	Value
Method	POST
URL	https://restful-booker.herokuapp.com/booking
Body type	Raw JSON
First name	{{fname}}
Other sample values	Biradar, 874, true, bookingdates, Breakfast and Meals
Tests	Save b_id; verify status 200; check body contains Abhi; check Content-Type; response time below 2000 ms

```
const jsonData = pm.response.json();
pm.environment.set("b_id", jsonData.bookingid);

pm.test("Verify Status Code is 200", function () {
    pm.response.to.have.status(200);
});

pm.test("Verify Body matches string", function () {
    pm.expect(pm.response.text()).to.include("Abhi");
});
```

```
pm.test("Content-Type is present", function () {
    pm.response.to.have.header("Content-Type", "application/json; charset=utf-8");
});

pm.test("Response time is less than 2000ms", function () {
    pm.expect(pm.response.responseTime).to.be.below(2000);
});
```

Line-by-line: the first line parses the response. The next line reads bookingid and saves it into the active environment as b_id. The tests then validate status, body text, a response header, and elapsed time.

Potential brittleness in the uploaded script The request body uses {{fname}}, while the test hard-codes Abhi. This is consistent with the supplied environment value, but if fname changes to another name, the body assertion fails even if the API correctly returns the new value. A better approach is to store the expected name in a variable or assert against the actual test input.

14.3 GET Booking Details As Per Id

Setting	Value
Method	GET
URL	https://restful-booker.herokuapp.com/booking/{{b_id}}
Assertions	Status 200; response text contains Abhi; firstname equals Abhi; totalprice greater than 800; nested check-in date equals 2016-04-22

```
const jsonData = pm.response.json();

pm.test("Verify Status Code 200", function () {
    pm.response.to.have.status(200);
});

pm.test("Verify First Name", function () {
    pm.expect(jsonData.firstname).to.eql("Abhi");
});

pm.test("Verify Total Price Greater Than 800", function () {
    pm.expect(jsonData.totalprice).to.be.greaterThan(800);
});

pm.test("Verify Checkin Date", function () {
    pm.expect(jsonData.bookingdates.checkin).to.eql("2016-04-22");
});
```

This request demonstrates exact string comparison, numeric comparison, and nested-object access in the same test script. It uses the b_id variable instead of a fixed numeric ID, making the request reusable when the ID is populated by Create Booking.

14.4 Token Generator

Setting	Value
Method	POST
URL	https://restful-booker.herokuapp.com/auth
Body	Username admin and password password123 in the demo request

Setting	Value
Test/action	Parse response; save token as b_token; verify status 200

```
const jsonData = pm.response.json();
pm.environment.set("b_token", jsonData.token);

pm.test("Verify Status Code is 200", function () {
    pm.response.to.have.status(200);
});
```

The script reads token from the authentication response and saves it into the selected environment. The Update and Delete requests then refer to that value in their Cookie header. Use only approved demo credentials in a training environment; never copy this pattern with real secrets committed into a shared collection.

14.5 Update Booking

Setting	Value
Method	PUT
URL	https://restful-booker.herokuapp.com/booking/{{b_id}}
Headers	Content-Type: application/json; Accept: application/json; Cookie: token={{b_token}}
Body	Full booking representation; first name Sneha; last name Karajagi; price 874; dates and additional needs
Test	Status code 200

```
pm.test("Status Code is 200", function () {
    pm.response.to.have.status(200);
});
```

PUT is used here to update the booking identified by b_id. The Cookie header provides the token captured earlier. The request body contains the replacement booking fields. The supplied collection only checks status 200; a stronger flow should call GET afterward and verify firstname is Sneha, lastname is Karajagi, and the other updated fields match.

14.6 Curl Command

The collection also contains a request named Curl Command. It is a POST request created from the cURL example. It sends a static booking payload with firstname Abhi. It does not have a test script and does not save its returned booking ID into b_id. This is useful as an import demonstration, but it creates a second independent booking during a full collection run.

14.7 Delete Booking

Setting	Value
Method	DELETE
URL	https://restful-booker.herokuapp.com/booking/{{b_id}}
Header	Cookie: token={{b_token}}
Test	Status code is 201

```
pm.test("Status code is 201", function () {
    pm.response.to.have.status(201);
});
```

This request deletes the booking referenced by b_id using the stored token. The expected status 201 is the assertion configured in this supplied collection. Do not generalize this status to all APIs: follow the status code specified by the API under test. After deletion, an additional GET can verify the documented not-found behavior.

14.8 Negative Scenario - GET Booking Details Invalid ID

Setting	Value
Method	GET
URL	https://restful-booker.herokuapp.com/booking/48746
Test	Status code is 404

```
pm.test("Status code is 404", function () {
    pm.response.to.have.status(404);
});
```

This is a negative test: it deliberately requests a booking ID expected not to exist. It confirms that the API reports the missing record rather than returning a successful booking response.

14.9 Negative Scenario - Create Booking With Invalid Characters

The supplied request puts !@#$ in the totalprice field without quotes. That makes the JSON syntactically invalid. The request is designed to receive a 400 response. This specifically tests malformed JSON parsing, not every possible invalid-price business rule.

```
"totalprice": !@#$
```

To test a valid JSON document with a semantically invalid value, create a separate request such as totalprice = -10 or totalprice = "not-a-number" (only where those values are meaningful to the API contract). The expected response depends on the service validation rules and should be confirmed from its documentation.

15. Real-World QA Scenarios

15.1 Booking creation and verification

A travel application receives a reservation request. Create a booking, capture the booking ID, retrieve it, and validate guest name, price, check-in, and check-out. This checks that data submitted by the client is persisted and returned correctly.

15.2 Booking price boundary

A business rule requires a booking to cost more than Rs. 800. Test 799, 800, and 801. Check that values below or equal to the boundary are handled according to the requirement and that values above it pass.

15.3 Authentication and authorization

Run a protected update with a valid token, a missing token, and an invalid token. Confirm that only the valid authorized request is accepted. Keep tokens in a secure variable store and never include them in bug screenshots or public logs.

15.4 API regression run after deployment

After a new build is deployed to QA, run the collection in the correct environment. Review failures by request and test name. Separate actual application defects from stale data, expired tokens, incorrect environment selection, or dependency-order problems.

15.5 cURL reproduction for a defect

Suppose Postman returns a 400 response for a booking request. Generate a cURL command, remove all sensitive values, and attach it to the defect ticket. The developer can run the same request from a terminal and compare behavior.

15.6 Test-data cleanup

Create test records with identifiable test data, capture each returned ID, and clean up only records created by the test. When a request creates multiple records, store each identifier separately so cleanup does not leave records behind or delete the wrong record.

16. Best Practices

1. Use clear test names such as "First name matches created booking" rather than vague names such as "Test 1".
2. Validate status code and response body. A 200 alone does not prove business correctness.
3. Parse JSON once per script and reuse the parsed object within that script.
4. Check that required properties exist before reading deeply nested fields.
5. Validate data types before numeric comparisons.
6. Use exact equality for exact requirements and partial matching only when partial matching is intentional.
7. Keep expected data consistent with request data. Avoid hard-coded names that disagree with environment variables.
8. Use environment variables for environment-specific values and dynamic IDs; use collection variables for values shared across environments only when that scope is appropriate.
9. Generate fresh IDs and tokens during a run instead of relying on stale exported values.
10. Run dependent requests in the correct order and document their dependencies.
11. Isolate requests that create extra records; capture and clean up every created record.
12. Use response-time thresholds based on an agreed requirement and stable test environment. A single run is only a smoke check, not a full performance test.
13. Avoid overly strict response-header string comparisons if the API contract allows multiple valid content types or parameters.
14. Keep production secrets out of collection exports, environment exports, source control, screenshots, and bug trackers.
15. Use a dedicated test environment and non-production data for destructive operations such as DELETE.
16. Review Collection Runner output and investigate each failed assertion instead of relying only on the overall run status.

17. Common Mistakes and Troubleshooting

Symptom	Likely cause	What to check
URL contains {{b_id}} literally or is empty	No active environment, variable disabled, or request ran before Create Booking.	Select the Booking API environment; verify b_id exists; run the prerequisite request first.

Symptom	Likely cause	What to check
Cookie header contains {{b_token}} or an old token	Token variable not populated or wrong environment selected.	Run Token Generator; inspect b_token without exposing it; select the correct environment.
First-name assertion fails after changing fname	Assertion still expects Abhi while request now sends another name.	Make expected value dynamic or update the assertion to match the scenario.
JSON parsing fails	Response is not JSON or request returned an error body in another format.	Inspect response status and body before calling pm.response.json().
Nested field assertion errors	bookingdates or checkin is missing/null.	Assert parent and child properties exist before comparing the value.
Runner passes some requests but later requests fail	The run order or selected subset omitted a prerequisite.	Run the dependency chain in order; include token and booking creation steps.
Unexpected extra booking records	Curl Command request creates another booking.	Move it to a separate folder or capture and delete its own booking ID.
Exported environment does not work later	ID deleted, token expired, or stale values exported.	Regenerate token and booking ID as part of the run.
Header assertion fails although body is correct	The exact Content-Type includes different casing/charset/parameters.	Check actual header and assert the contract-supported media type rather than an unnecessarily rigid string.

18. Practice Exercises

Exercise 1 - First name

Write a test that verifies firstname is exactly Abhi. Add a property-existence check before the equality assertion.

Exercise 2 - Price boundaries

Write assertions for totalprice being a number and greater than 800. Try values 799, 800, and 801 and explain each result.

Exercise 3 - Nested date

Validate bookingdates.checkin and bookingdates.checkout. Add checks that bookingdates exists and both date fields are present.

Exercise 4 - cURL round trip

Export Create Booking to cURL, import the command into a separate folder, inspect the generated method/body/headers, and send it against a test environment.

Exercise 5 - Runner dependency

Run GET Booking Details without first running Create Booking. Observe the result when b_id is missing or stale. Then run Create Booking followed by GET Booking Details and compare.

Exercise 6 - Clean collection

Identify every request in the collection that creates a booking. Ensure each created booking ID is captured and cleaned up. Explain why the current Curl Command request is not cleaned up by the current Delete Booking request.

19. Interview Questions and Quick Revision

1. What is an API assertion, and why do we need it?
2. What does pm.response.json() do?
3. What is the role of pm.test()?
4. What is the difference between .eql() and .include()?
5. How do you validate a numeric field is greater than a threshold?
6. How do you access a nested JSON property such as bookingdates.checkin?
7. Why should you verify the data type before comparing numeric values?
8. What is cURL and why would a QA engineer use it?
9. How do you export a Postman request as cURL and import cURL into Postman?
10. What is Collection Runner and when would you use it?
11. Why does request execution order matter?
12. What is the difference between a collection export and an environment export?
13. Why are environment exports potentially sensitive?
14. How would you troubleshoot a missing {{b_id}} variable?
15. How would you verify that a PUT request actually updated a booking?

20. Final End-to-End Practice Flow

Use this as a repeatable lab. Select the Booking API environment before starting. The exact response status codes shown below are those expected by the uploaded collection; confirm against the demo API response if its behavior changes.

1. Run Token Generator and confirm the 200 test passes and b_token is populated.
2. Run Create Booking and confirm the 200 test passes and b_id is saved from the response bookingid.
3. Run GET Booking Details As Per Id and validate first name, total price, and nested check-in date.
4. Run Update Booking and verify the 200 status.
5. Run GET Booking Details again and validate the updated name and values. Update the expected assertions to the post-update test data.
6. Run Delete Booking and validate the configured 201 status.
7. Run GET for the deleted ID and verify the expected not-found response (if supported by the demo service).
8. Run the Negative Scenario requests independently and inspect their response bodies as well as status codes.
9. Run the cURL-derived request separately, capture its own booking ID, and clean it up.
10. Export the collection and a sanitized environment. Never include a live token in a shared export.

> ***Scope and safety:*** *Based on Flow(6).txt and the Booking API collection/environment exports. Demo IDs and tokens may expire; sanitize exports and never send confidential or production data to a public demo API.*